



1. ¿QUÉ ES UN ÍNDICE?

Un índice es una estructura de datos que SQL Server crea para mejorar la velocidad de búsqueda y recuperación de registros en una tabla.

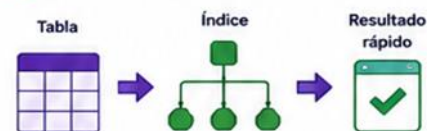
Funciona como el índice de un libro: permite encontrar rápidamente la información sin recorrer toda la tabla.



Idea clave: Los índices aceleran las consultas, pero pueden afectar el rendimiento de las inserciones, actualizaciones y eliminaciones.

2. ¿PARA QUÉ SIRVE?

- ✓ Acelerar las consultas SELECT.
- ✓ Mejorar el rendimiento en JOIN.
- ✓ Optimizar las búsquedas y filtros (WHERE).
- ✓ Ordenar datos para consultas ORDER BY y GROUP BY.
- ✓ Garantizar la unicidad de valores (con índices únicos).
- ✓ Reducir el I/O (lecturas de disco).



3. ¿CUÁNDO USAR ÍNDICES?

- 🔍 Cuando las consultas usan frecuentemente columnas en WHERE, JOIN, ORDER BY, GROUP BY.
- 📊 En columnas con muchos registros y pocas repeticiones.
- 🔑 Para garantizar unicidad de datos (Índices únicos).
- 📄 En tablas grandes que reciben muchas consultas de lectura.
- 🔄 Cuando el rendimiento de consultas es lento.

4. TIPOS DE ÍNDICES EN SQL SERVER



Índice Clusterizado (CLUSTERED)

Define el orden físico de los datos en la tabla. Solo puede existir uno por tabla.

Ejemplo: `CREATE CLUSTERED INDEX IX_Cliente ON Clientes(IdCliente);`



Índice No Clusterizado (NONCLUSTERED)

Crea una estructura separada de la tabla. Puede haber varios por tabla.

Ejemplo: `CREATE NONCLUSTERED INDEX IX_Nombre ON Clientes(Nombre);`



Índice Único (UNIQUE)

No permite valores duplicados en la columna o combinación de columnas.

Ejemplo: `CREATE UNIQUE INDEX IX_Email ON Clientes(Email);`



Índice de Texto Completo (FULLTEXT)

Permite buscar texto dentro de columnas de tipo carácter.

Ejemplo: `CREATE FULLTEXT INDEX ON Articulos (Description LANGUAGE 1033);`



Índice Columnstore

Optimizado para consultas analíticas en grandes volúmenes de datos.

Ejemplo: `CREATE CLUSTERED COLUMNSTORE INDEX CCI_Ventas ON Ventas;`

5. SINTAXIS BÁSICA

Crear índice Clusterizado

```
CREATE CLUSTERED INDEX NombreIndice
ON Tabla (Columna);
```



Crear índice No Clusterizado

```
CREATE NONCLUSTERED INDEX NombreIndice
ON Tabla (Columna);
```



Crear índice Único

```
CREATE UNIQUE INDEX NombreIndice
ON Tabla (Columna);
```



Crear índice en múltiples columnas

```
CREATE NONCLUSTERED INDEX NombreIndice
ON Tabla (Columna1, Columna2, Columna3);
```



Eliminar índice

```
DROP INDEX NombreIndice ON Tabla;
```



6. EJEMPLOS PRÁCTICOS

1) Índice Clusterizado

```
CREATE CLUSTERED INDEX IX_Clientes_IdCliente
ON Clientes(IdCliente);
-- Define el orden físico por IdCliente
```



2) Índice No Clusterizado

```
CREATE NONCLUSTERED INDEX IX_Clientes_Nombre
ON Clientes(Nombre);
-- Acelera búsquedas por nombre
```



3) Índice Único

```
CREATE UNIQUE INDEX IX_Clientes_Email
ON Clientes(Email);
-- Garantiza que el email sea único
```



4) Índice en múltiples columnas

```
CREATE NONCLUSTERED INDEX IX_Ventas_FechaCliente
ON Ventas(FechaVenta, IdCliente);
-- Útil para filtros por fecha y cliente
```



5) Eliminar índice

```
DROP INDEX IX_Clientes_Nombre ON Clientes;
```



7. VER ÍNDICES DE UNA TABLA

Consultar los índices existentes:

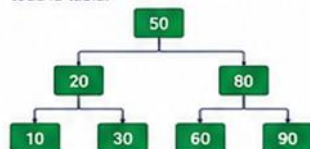
```
EXEC sp_helpindex 'NombreTabla';
```

```
SELECT * FROM sys.indexes
WHERE object_id = OBJECT_ID('NombreTabla');
```



8. CÓMO FUNCIONAN LOS ÍNDICES

SQL Server utiliza un índice (generalmente B-Tree) para localizar rápidamente las filas sin tener que escanear toda la tabla.



- Búsqueda rápida
- Menos lecturas de disco
- Mejor rendimiento en consultas

9. VENTAJAS Y DESVENTAJAS

VENTAJAS

- ✓ Consultas más rápidas.
- ✓ Mejora el rendimiento general.
- ✓ Reduce lecturas de disco.
- ✓ Optimiza JOIN, WHERE, ORDER BY y GROUP BY.
- ✓ Soporta índices únicos para integridad de datos.

DESVENTAJAS

- ✗ Ocupan espacio en disco adicional.
- ✗ Disminuyen el rendimiento en INSERT, UPDATE y DELETE.
- ✗ Requieren mantenimiento (reconstrucción).
- ✗ Demasiados índices pueden afectar el rendimiento.

10. MANTENIMIENTO DE ÍNDICES

Fragmentación: Los índices se fragmentan con el tiempo.
Solución: Reorganizar o reconstruir.

Reorganizar índice

```
ALTER INDEX NombreIndice
ON Tabla REORGANIZE;
```



Reconstruir índice

```
ALTER INDEX NombreIndice
ON Tabla REBUILD;
```



11. EJEMPLO COMPLETO

Tabla: Clientes

IdCliente	Nombre	Email
1	Ana López	ana@mail.com
2	Luis Pérez	luis@mail.com
3	Maria Gómez	maria@mail.com
4	Carlos Ruiz	carlos@mail.com
5	Laura Díaz	laura@mail.com

Creamos índices:

- Índice Clusterizado en IdCliente
- Índice No Clusterizado en Nombre
- Índice Único en Email

Consultas optimizadas:

```
-- Búsqueda por Nombre
SELECT * FROM Clientes
WHERE Nombre = 'María Gómez';
```

```
-- Búsqueda por Email (único)
SELECT * FROM Clientes
WHERE Email = 'luis@mail.com';
```

```
-- Orden por Nombre
SELECT * FROM Clientes
ORDER BY Nombre;
```



12. BUENAS PRÁCTICAS

- ✓ Crear índices en columnas usadas frecuentemente en consultas y filtros.
- ✓ Evitar índices en columnas con muchos cambios.
- ✓ No crear índices innecesarios.
- ✓ Usar índices únicos cuando aplique.
- ✓ Monitorear y mantener índices periódicamente.
- ✓ Revisar planes de ejecución para validar uso de índices.



COMANDOS PRINCIPALES

```
CREATE INDEX
ALTER INDEX
DROP INDEX
sp_helpindex
sys.indexes
```



RESUMEN RÁPIDO



Tabla

Índice

Búsqueda rápida

Mejor rendimiento



RECUERDA

Los índices son herramientas poderosas, pero deben usarse con criterio para obtener el mejor rendimiento.

